

WEEK-01

BASICS- MACHINE LEARNING

CONTENTS

- ❖ COMMA SEPARATED VALUES (CSV)
- ❖ CSV FILE IN PYTHON
- ❖ CSV FILES WITH PANDAS
- ❖ IMPORTANT LIBRARIES

Why CSV files are useful for Machine Learning

- CSV files are widely used for storing tabular data.
- They are a ***simple and efficient way to organize data in a plain text format***, making them compatible with various software applications, including Microsoft Excel.
- Furthermore, CSV files can be ***easily edited and manipulated using spreadsheet software***, allowing you to make quick changes to your data without the need for complex coding or programming.
- CSV files serve a number of different purposes.
- They help users export a high volume of data to a more concentrated database.
- They also have following advantages:
 - ✓ CSV files are plain-text files, making them easier for users to create.
 - ✓ Since they're plain text, they're easier to import into a spreadsheet or another storage database, regardless of the specific software you're using.
 - ✓ It better organizes large amounts of data.

How to Create a CSV File?

- A CSV is a text file, so it can be created and edited using any text editor (like Notepad).
- More frequently, however, a CSV file is created by exporting (File > Export) a spreadsheet or database in the program that created it.

➤ Using Notepad

- ✓ To create a CSV file with a text editor, first choose your favourite text editor, such as Notepad, and open a new file.
- ✓ Then enter the text data you want the file to contain, separating each value with a comma and each row with a new line.

```
Title1,Title2,Title3
one,two,three
example1,example2,example3
```

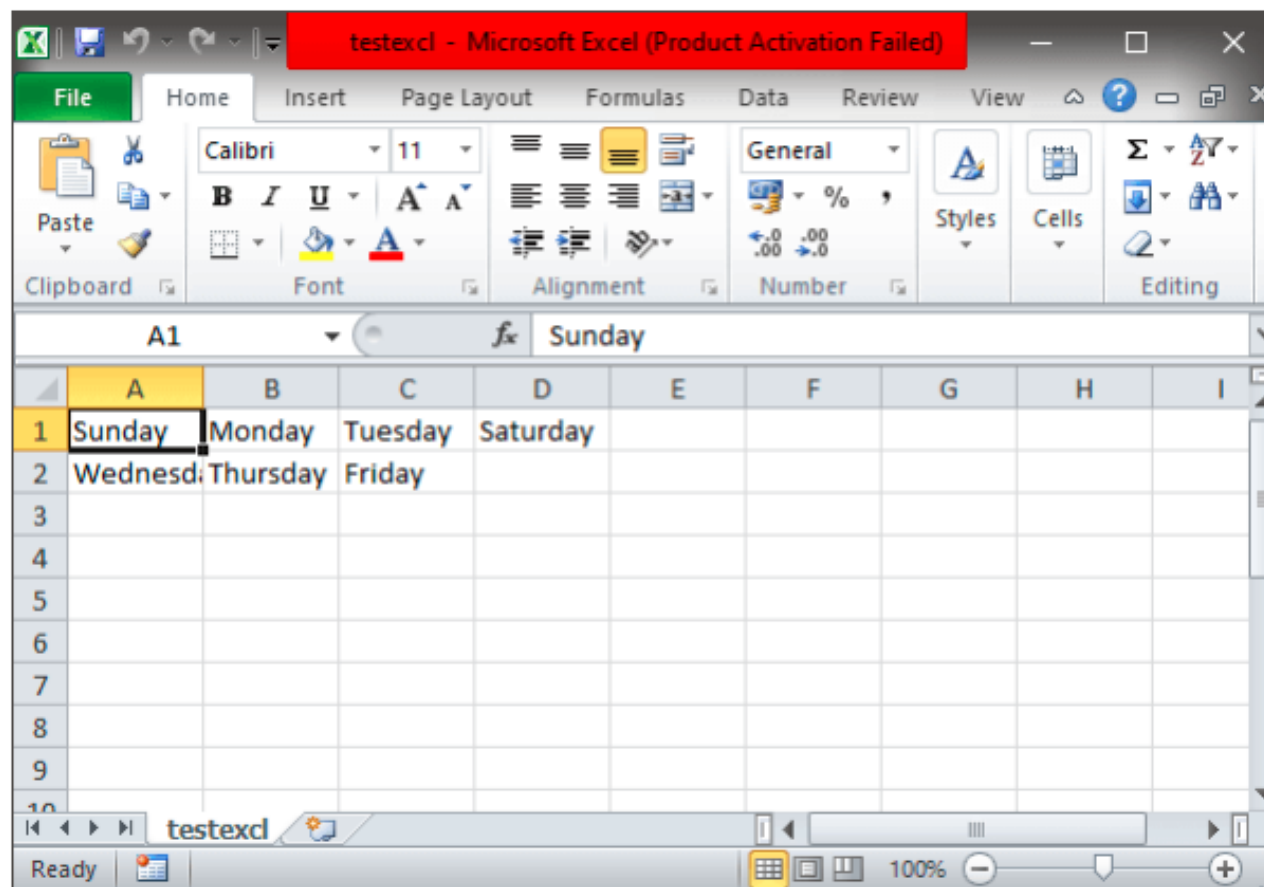
- *Save this file with the extension .csv.*
- *You can then open the file using Microsoft Excel or any other spreadsheet program.*

It would create a table of data similar to the following:

Title1	Title2	Title3
one	two	three
example1	example2	example3

Using Spreadsheet

- To create a CSV file using spreadsheet software (like Microsoft Excel), launch the program and then enter the data in cells (each value in a separate cell and each row with a new row).
- After entering the data click **File** and choose **Save As**. Under **Save as type**, select **Text CSV** and save the file.



CSV FILE IN PYTHON

Working With CSV File in Python

- For working CSV files in python, there is an inbuilt module named csv.
- Python provides a CSV module to handle CSV files.
- To read/write data, you need to loop through rows of the CSV.
- The **csv module** implements classes *to read and write tabular data in CSV format*.
- It allows programmers to say "write the data in the format preferred by Excel", or "read data from this file which was generated by Excel", without knowing the precise details of the CSV format used by Excel.
- *Programmers can also describe the CSV formats understood by other applications or define their own special-purpose CSV formats.*
- To pull the information from CSV files you use loop and split methods to get the data from individual columns.
- The CSV module explicitly exists to handle this task, making it much easier to deal with CSV formatted files.
- This becomes especially important when you are working with data that's been exported from actual spreadsheets and databases to text files.
- The information can be tough to read on its own.

CSV FILE IN PYTHON

Reading CSV Files

- To read data from a CSV file, you must use the **reader** function to generate a reader object.
- The reader function is developed to take each row of the file and make a list of all columns.
- Then, you have to choose the column you want the variable for.
- To understand it better, let's consider the following example. Let the data.csv contains the following data:
 - Programming language, Designed by, Appeared, Extension
 - Python, Guido van Rossum, 1991, .py
 - Java, James Gosling, 1995, .java
 - C++, Bjarne Stroustrup, 1983, .cpp

```
#import necessary modules
import csv
with open('/home/student/Documents/data.csv', 'rt') as f:
    data=csv.reader(f)
    for row in data:
        print(row)
```



When you execute the program above, the output will be:

```
['Programming language; Designed by; Appeared; Extension']
['Python; Guido van Rossum; 1991; .py']
['Java; James Gosling; 1995; .java']
['C++; Bjarne Stroustrup;1983;.cpp']
```

CSV FILE IN PYTHON

Reading CSV file as a Dictionary

- You can also use **DictReader** to read CSV files.
- The results are interpreted as a dictionary where the header row is the key, and the other rows are the values.

```
#import necessary modules
import csv
reader=csv.DictReader(open("/home/student/Documents/data.csv",'rt'))
for row in reader:
    print(row)
```



The result of this code is:

```
OrderedDict([('Programming language', 'Python'), ('Designed by', 'Guido van Rossum'), (' Appeared', ' 1991'), (' Extension', ' .py')])
OrderedDict([('Programming language', 'Java'), ('Designed by', 'James Gosling'), (' Appeared', ' 1995'), (' Extension', ' .java')])
OrderedDict([('Programming language', 'C++'), ('Designed by', ' Bjarne Stroustrup'), (' Appeared', ' 1985'), (' Extension', ' .cpp')])
```


CSV FILE IN PYTHON

Writing CSV Files

- When you have a set of data that you would like to store in a CSV file you have to use the **writer()** function.
- To iterate the data over the rows, you have to use the **writerow()** function.
- Consider the following example. We write the data into a file "writeData.csv" where the delimiter is an apostrophe.

```
#import necessary modules
import csv
with open ("/home/student/Documents/data1.csv",mode='w') as file:
    writer=csv.writer(file,delimiter=',',quotechar='"',quoting=csv.QUOTE_MINIMAL)
    writer.writerow(['Programming Language', 'Designed by', 'Appeared','Extension'])
    writer.writerow(['Python', 'Guido Van Rossam', '1991','.py'])
    writer.writerow(['Java', 'James Gosling', '1995','.java'])
    writer.writerow(['C++', 'Bjarne Stroupstrup', '1985','.cpp'])
```

Result in the csv file is:

```
Programming language, Designed by, Appeared, Extension
Python, Guido van Rossum, 1991, .py
Java, James Gosling, 1995, .java
C++, Bjarne Stroustrup, 1983, .cpp
```

CSV FILES WITH PANDAS

Reading CSV Files with Pandas

- Pandas is an open-source library that allows you to import CSV in Python and perform data manipulation.
- Pandas provide an easy way to create, manipulate and delete data.
- Reading the CSV into a pandas DataFrame is very quick and easy:

```
#import necessary modules
import pandas
result = pandas.read_csv('/home/student/Documents/data1.csv')
print(result)
```

The result of the above will be:

Programming language, Designed by, Appeared, Extension

0 Python, Guido van Rossum, 1991, .py

1 Java, James Gosling, 1995, .java

2 C++, Bjarne Stroustrup, 1983, .cpp

- It's very easy to read CSV files using pandas.
- In just three lines of code, you get the same result as earlier.
- Pandas know that the first line of the CSV contains column names, and it will use them automatically.

CSV FILES WITH PANDAS

Writing CSV Files With Pandas

- Writing to a CSV file with Pandas is as easy as reading.
- First, you must create a DataFrame and write the data in it.
- Then export the data from the DataFrame to a CSV file.
- The following code does the same:

```
from pandas import DataFrame
C={'Programming Language':['Python','Java','C++'],
  'Designed by':['Guido Van Rossam','James Gosling','Bjarne Stroustrup'],
  'Appeared':['1991','1995','1985'],
  'Extension':['.py','.java','.cpp'],
  }
df=DataFrame(C,columns=['Programming Language','Designed by','Appeared','Extension'])
export_csv=df.to_csv(r'/home/student/Documents/data2.csv',index=None,header=True)
print(df)
```

The output of the above code will be the following CSV file:

	Programming language	Designed by	Appeared	Extension
0	Python	Guido van Rossum	1991	.py
1	Java	James Gosling	1995	.java
2	C++	Bjarne Stroustrup	1983	.cpp

CSV FILES WITH PANDAS

- So, now you know how to use the method 'csv' and also read and write data in CSV format.
- CSV files are widely used in software applications because they are easy to read and manage, and their small size makes them relatively fast for processing and transmission.
- The csv module provides various functions and classes which allow you to read and write easily.
- CSV is the best way for saving, viewing, and sending data. Pandas is also a great alternative to reading CSV files.
- Actually, it isn't as hard to learn as it seems at the beginning. But with a little practice, you'll master it.

IMPORTANT LIBRARIES

- ✓ NumPy
- ✓ Pandas
- ✓ SciPy
- ✓ Matplotlib
- ✓ Scikit Learn.
- ✓ Seaborn

Install

```
$pip install numpy  
$pip install pandas  
$pip install scipy  
$pip install matplotlib  
$pip install seaborn  
$pip install scikit-learn
```

Check at UBUNTU

Open terminal and type **spyder**

At Console type

```
import numpy  
import pandas  
import scipy  
import matplotlib  
import seaborn  
import sklearn
```

Thank You